RETAIL INTELLIGENCE PLATFORM

Store Intelligence Architecture

Transforming Raw CCTV Streams into Real-Time, Structured Retail Analytics

PROBLEM STATEMENT

The Retail "Dark Data" Challenge

Retail stores generate **petabytes of CCTV footage** every year, yet **99% of it goes unused and unanalyzed**. This represents a massive untapped reservoir of operational intelligence – visitor behavior, dwell patterns, and conversion signals buried in unstructured video pixels.

Unique Visitor Counts

Accurately track individual footfall across all store zones without double-counting re-entries.

Zone Engagement Maps

Map real-time visitor density across product categories to identify high-traffic and underperforming areas.

Queue Bottleneck Analysis

Detect and measure checkout congestion to optimize staffing and reduce customer wait times.

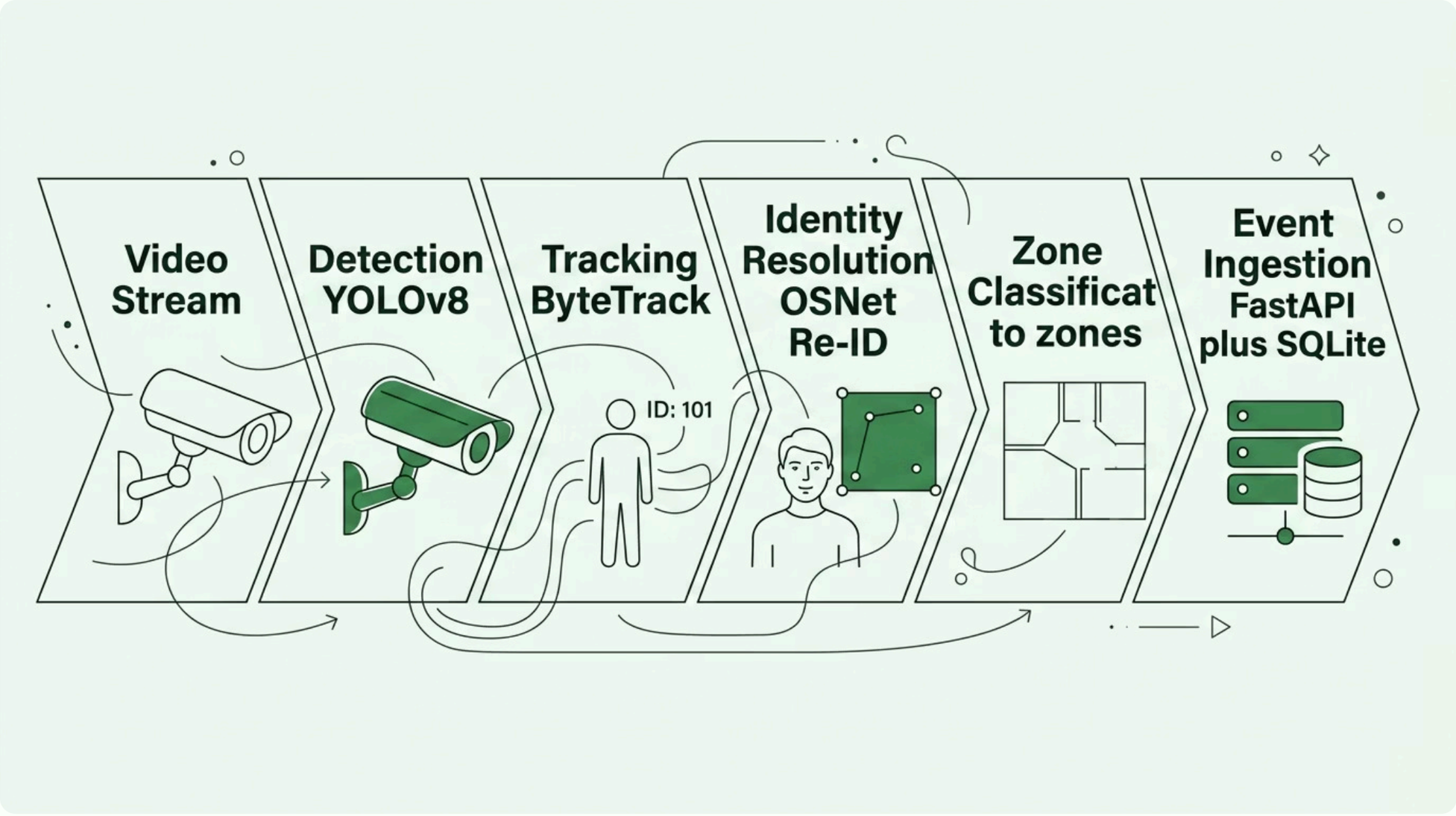
Purchase Conversion Funnels

Correlate zone visits with purchase events to build end-to-end conversion analytics.

SYSTEM ARCHITECTURE

End-to-End Edge Architecture

The pipeline is engineered as a sequence of **decoupled, swappable components** – each responsible for a single transformation stage. This modular design ensures that YOLOv8, ByteTrack, or SQLite can be replaced in the future without rebuilding the core engine.



Each stage operates independently, communicating through well-defined interfaces. The pipeline processes video frames in real time at the edge, producing structured business events ready for downstream analytics and reporting.

DETECTION & TRACKING

Selecting the Foundation: YOLOv8 & ByteTrack

Component selection prioritized the intersection of **real-time performance, ecosystem maturity, and edge deployability**. Both components were evaluated against leading alternatives before final selection.

YOLOv8 — Object Detection

Evaluated against: RT-DETR, YOLOv9

Chosen for its unmatched balance of inference speed, model maturity, and a rich developer ecosystem. YOLOv8 deploys cleanly on edge GPU hardware with minimal configuration overhead.

- Best-in-class speed-to-accuracy ratio
- Mature, well-documented Ultralytics ecosystem
- Native ONNX and TensorRT export support

ByteTrack — Multi-Object Tracking

Evaluated against: DeepSORT

Selected for exceptional real-time crowd handling with low computational overhead. ByteTrack's association logic pairs seamlessly with YOLOv8's detection output, avoiding the identity-switch problems common in Kalman-filter-based trackers.

- Superior performance in dense crowd scenarios
- No separate Re-ID model required for basic tracking
- Proven synergy and compatibility with YOLOv8

IDENTITY RESOLUTION

Visitor Identity Resolution at the Edge

ByteTrack assigns **local, temporary track IDs** that reset when a camera disconnects or a visitor re-enters the scene. To preserve continuity across cameras and visits, the edge pipeline creates a stable **visitor_id** from appearance-based Re-ID embeddings.

OSNet Re-ID Embeddings — Selected

How it works: each detected person is passed through OSNet Re-ID to extract visual features such as clothing color, silhouette, and gait, then encoded as a **256-dimensional embedding vector**.

The system compares each new embedding against a rolling window of recent embeddings using **cosine similarity**. When similarity exceeds the matching threshold, the same visitor_id is reused across camera zones and re-entry events.

- Stable cross-camera identity without relying on ByteTrack track IDs
- Accurate unique visitor counts with double-count prevention
- Tracks customer journey across store zones and return visits
- Appearance-based only: no face recognition, no biometric identifiers

Facial Recognition — Rejected

Facial recognition was excluded because the architecture must remain **privacy-first** and deployment-friendly at the edge. It introduces biometric processing, higher consent and governance overhead, and dependency on clear face visibility that is often unavailable in retail or crowded environments.

- Requires biometric data and stronger privacy controls
- Fails when faces are occluded, turned away, or out of frame
- More sensitive to legal, operational, and trust concerns
- Not needed when appearance embeddings solve the identity problem

SPATIAL INTELLIGENCE

Deterministic Polygon-Based Zones

Spatial zone classification maps each visitor's **foot coordinates** directly into arbitrary, multi-vertex polygons aligned to physical store layouts – such as `SKINCARE`, `MAKEUP`, and `BILLING_QUEUE`.

✅ Polygon Zoning — Selected

Polygons are defined once in a configuration file, mapped to camera coordinate space, and evaluated via a point-in-polygon test on every tracked foot position.

- **100% deterministic** – identical input always produces identical output
- **Fully explainable** – zone boundaries are human-readable and auditable
- **Sub-millisecond** evaluation with zero ML inference cost

❌ VLM Semantic Zoning — Rejected

Vision-Language Models were evaluated as an alternative for AI-suggested semantic zone detection. Despite initial promise, this approach was rejected.

- **Extreme computational cost** – impractical on edge hardware
- **High latency** – incompatible with real-time processing requirements
- **Poor accuracy** on low-resolution CCTV frames
- **Non-deterministic** outputs complicate auditing and debugging

DATA LAYER

Why SQLite Outperforms PostgreSQL at the Edge

The ingestion layer evaluated both SQLite and PostgreSQL for persisting structured analytics events. SQLite was selected after a rigorous comparison against the operational realities of single-site, edge-deployed workloads.

Criteria	SQLite — Selected ✅	PostgreSQL — Rejected ❌
Operational Overhead	Zero – self-contained, file-based, no server process	High – requires clustering, connection pooling, and management
Hardware Footprint	Extremely lightweight – ideal for constrained edge nodes	Heavy resource demand – memory and CPU intensive
Deployment Complexity	Single file – no external dependencies or configuration	Requires dedicated database container and initialization
Workload Suitability	Perfectly matched to single-site store event volumes	Overkill – designed for multi-tenant, high-concurrency workloads
Recovery & Portability	Database file can be copied, backed up, or migrated instantly	Requires dump/restore procedures for migration

DEPLOYMENT

Decoupled Containers for Speed and Scale

The architecture is split into **two specialized Docker images**, each optimized for a distinct role. This separation enables independent scaling, faster iteration, and dramatically improved startup behavior.

Dockerfile.pipeline — Heavy ML Container (~4 GB)

Bundles all heavyweight ML frameworks: **PyTorch, OpenCV, Ultralytics (YOLOv8), and OSNet Re-ID**. This GPU-capable container handles the full video processing pipeline. Initialization is slower due to framework loading, but it runs continuously once warm.

Dockerfile.api — Lightweight API Container

A minimal Python container running **FastAPI and SQLite**. Boots in seconds to immediately serve historical analytics queries and health checks – even while the heavy pipeline container is still initializing in the background.

Operational Value

This decoupling ensures the **API layer is always available** for dashboards and monitoring, regardless of pipeline state. Rolling updates to the ML stack can be performed without interrupting the analytics API surface.

EDGE OPERATIONS

Handling Real-World Edge Anomalies

Edge deployments face unpredictable conditions – network blips, container restarts, and overlapping camera coverage. The architecture includes three targeted heuristics to maintain data integrity under these conditions.

Staff Exclusion Heuristics

Staff are filtered dynamically based on **spatial behavior patterns, trajectory loops, and dwell-time thresholds** – rather than relying on clothing classifiers that are easily fooled by uniform changes. This behavioral approach is far more robust in production.

FastAPI Ingestion Idempotency

The ingestion endpoint uses **SQL**

ARCHITECTURE SUMMARY

Core Design Principles

Every architectural decision in this system traces back to four foundational pillars. These principles guided component selection, data layer choices, and operational heuristics throughout the build.

Reliability

Built to withstand local edge pipeline restarts without losing state. Idempotent ingestion and Redis-backed deduplication ensure metrics remain accurate through any failure scenario.

Explainability

Clear, deterministic spatial zones replace black-box ML decisions. Every zone assignment and visitor count can be traced, audited, and validated by non-technical stakeholders.

Simplicity

Pragmatic SQLite and YOLOv8 integrations reduce development and maintenance overhead. The system favors proven, well-documented tools over novel but fragile alternatives.

Practicality

Operational heuristics prioritize real-world physical constraints over theoretical complexity. Staff exclusion, deduplication, and container decoupling solve actual production problems.

Architecture Outcome: A production-grade, edge-deployable retail intelligence system that transforms raw CCTV streams into structured, real-time business analytics – with zero dark data.